

Examen d'entraînement LEPL1110 — Éléments Finis

Claude code

Mai 2026

Durée : 3 heures. Aucun document autorisé. Calculatrice non autorisée.

Les exercices sont indépendants et peuvent être traités dans n'importe quel ordre.

Exercice 1 — Poutre d'Euler-Bernoulli et éléments de Hermite

On considère la flèche $w(x)$ d'une **poutre encastrée-libre** de longueur $L = 1$, de rigidité $EI = 1$, soumise à une charge uniforme $q > 0$:

$$EI w''''(x) = q, \quad x \in (0, 1),$$

avec

$$w(0) = 0, \quad w'(0) = 0 \quad (\text{encastrement}), \quad EI w''(1) = 0, \quad EI w'''(1) = 0 \quad (\text{extrémité libre}).$$

1.1 — Formulation faible

Soit $V = \{v \in H^2(0, 1) : v(0) = 0, v'(0) = 0\}$.

En multipliant l'équation par $v \in V$ et en intégrant **deux fois** par parties, montrer que :

$$\text{Trouver } w \in V \text{ tel que } \int_0^1 w'' v'' dx = \int_0^1 q v dx, \quad \forall v \in V.$$

Préciser quelles conditions sont **essentiels** et lesquelles sont **naturelles**.

1.2 — Éléments de Hermite

Pour un élément $[x_e, x_{e+1}]$ de longueur h , avec $\xi = (x - x_e)/h \in [0, 1]$, les quatre fonctions de forme de Hermite sont :

$$\varphi_1(\xi) = 1 - 3\xi^2 + 2\xi^3, \quad \varphi_2(\xi) = h(\xi - 2\xi^2 + \xi^3), \quad \varphi_3(\xi) = 3\xi^2 - 2\xi^3, \quad \varphi_4(\xi) = h(-\xi^2 + \xi^3).$$

DDLs : $\mathbf{u}^{(e)} = (w_e, \theta_e, w_{e+1}, \theta_{e+1})^\top$ où $\theta = w'$.

(a) Vérifier les conditions d'interpolation : $\varphi_1(0) = 1, \frac{d\varphi_2}{dx}\big|_{\xi=0} = 1, \varphi_3(1) = 1, \frac{d\varphi_4}{dx}\big|_{\xi=1} = 1$, et que toutes les autres conditions nodales valent zéro.

(b) Calculer les **dérivées secondes** $\varphi_i''(\xi) = d^2\varphi_i/d\xi^2$.

1.3 — Matrice de rigidité élémentaire

(a) Avec le changement de variable $\xi = (x - x_e)/h$, montrer que :

$$K_{11}^{(e)} = \frac{EI}{h^3} \int_0^1 [\varphi_1''(\xi)]^2 d\xi, \quad K_{12}^{(e)} = \frac{EI}{h^2} \int_0^1 \varphi_1''(\xi) \varphi_2''(\xi) d\xi.$$

(b) Calculer $K_{11}^{(e)}$ et $K_{12}^{(e)}$.

(c) En utilisant les symétries, écrire la matrice de rigidité complète :

$$K^{(e)} = \frac{EI}{h^3} \begin{pmatrix} 12 & 6h & -12 & 6h \\ 6h & 4h^2 & -6h & 2h^2 \\ -12 & -6h & 12 & -6h \\ 6h & 2h^2 & -6h & 4h^2 \end{pmatrix}.$$

Vérifier $K^{(e)}(1, 0, 1, 0)^\top = \mathbf{0}$ et interpréter.

1.4 — Application : $N = 1$ élément, charge uniforme q

$N = 1$ élément de Hermite sur $[0, 1]$ ($h = 1$, $EI = 1$). DDLs : $u_1 = w(0)$, $u_2 = \theta(0)$, $u_3 = w(1)$, $u_4 = \theta(1)$.

(a) Calculer le vecteur charge $F_i = q \int_0^1 \varphi_i(\xi) d\xi$.

(b) Après imposition de $u_1 = u_2 = 0$, écrire le système réduit $K_r(u_3, u_4)^\top = (F_3, F_4)^\top$ et calculer u_3 et u_4 .

(c) La solution exacte est $w(x) = \frac{q}{24}(6x^2 - 4x^3 + x^4)$. Comparer u_3 , u_4 aux valeurs exactes $w(1)$ et $w'(1)$. Justifier.

1.5 — Convergence

(a) Les éléments de Hermite sont cubiques ($k = 3$). Quelle est la vitesse de convergence de $\|w - w_h\|_{H^2}$?

(b) Pourquoi les éléments de Hermite nécessitent-ils une continuité C^1 entre éléments (contrairement aux éléments de Lagrange P_k) ?

Exercice 2 — Le jeu des erreurs

La fonction ci-dessous est censée calculer la **norme d'erreur** L^2 entre une solution exacte u_{ex} et une solution éléments finis P_1 (valeurs nodales u_{nodes}) par quadrature de Gauss–Legendre à 2 points.

Ce code contient volontairement un maximum de sept erreurs.

```
1 import numpy as np
2
3 def compute_l2_error(u_exact, u_nodes, N):
4     """
5     Norme d'erreur L2 entre u_exact et la solution P1 FEM.
6     u_nodes : tableau de N+1 valeurs nodales sur (0,1).
7     """
8     h = 1.0 / N
9     error_sq = 0.0
10
11     xi_q = [-1.0 / np.sqrt(3.0), 1.0 / np.sqrt(3.0)]
12     w_q = [1.0, 1.0]
13
14     for e in range(N + 1):
15         errone
16
17         x_left = e * h
18         x_right = (e + 1) * h
19
20         for q in range(2):
21             xi = xi_q[q]
22             w = w_q[q]
```

```

23         x_q = 0.5 * (x_left + x_right) + xi      # a annoter si
           errone
24
25         N1 = 0.5 * (1.0 + xi)                    # a annoter si
           errone
26         N2 = 0.5 * (1.0 - xi)                    # a annoter si
           errone
27
28         u_h = N1 * u_nodes[e] + N2 * u_nodes[e + 1]
29
30         diff = u_exact(x_q) - u_h
31         error_sq += diff**2 * w                    # a annoter si
           errone
32
33         error_sq = error_sq / N                    # a annoter si
           errone
34
35     return error_sq                                # a annoter si
           errone

```

Liste des erreurs (à compléter) :

- ① _____
- ② _____
- ③ _____
- ④ _____
- ⑤ _____
- ⑥ _____
- ⑦ _____